

category: avalonia

tags: [keyboard, telex, vietnamese, ime, virtual-keyboard, kzkeyboard, styledproperty, persistence]

severity: high

created: 2026-07-30

updated: 2026-07-30

project-origin: MolexAPP (MolexKioskApp)

KzKeyboard: `TelexEnabled` (StyledProperty public) hoàn toàn tách rời khỏi phím TELEX thật — host app không bao giờ quan sát được thay đổi

Tình huống gặp phải

Cần lưu lại ngôn ngữ bàn phím ảo (VIE/ENG) mỗi khi user bấm nút toggle trên `KzKeyboard`, để lần mở lại (view khác hoặc mở lại app) tự động load đúng ngôn ngữ đã chọn. Cách làm: set `keyboard.TelexEnabled = saved` lúc khởi tạo, và subscribe `keyboard.PropertyChanged` để lưu lại mỗi khi `TelexEnabledProperty` đổi — pattern chuẩn cho `AvaloniaProperty` public.

Triệu chứng / Lỗi

Build pass, không exception. Nhưng: bấm nút VIE/ENG trên bàn phím ảo → UI đổi đúng (label, highlight, hành vi gõ Telex đều đổi) nhưng **file settings không bao giờ được ghi** — có nghĩa `PropertyChanged` của `TelexEnabledProperty` không bao giờ fire khi user thao tác thật, dù code gán `keyboard.TelexEnabled = value` từ code-behind (lúc load) vẫn hoạt động bình thường.

Nguyên nhân gốc rễ (Root Cause)

`KzKeyboard.cs` có 2 biến trạng thái riêng biệt cho cùng 1 khái niệm:

```
public static readonly StyledProperty<bool> TelexEnabledProperty = ...; // public, bindable
public bool TelexEnabled { get => GetValue(...); set { SetValue(...); ... } } // wrapper đúng

private bool _telexEnabled = false; // ← field riêng, KHÔNG liên quan StyledProperty
```

`DrawKey` (vẽ highlight + label) và `ProcessKey` (xử lý phím bấm thật) đều đọc/ghi

`_telexEnabled` — field private — chứ không đung đến `TelexEnabledProperty`:

```
case "TELEX":
    _telexEnabled = !_telexEnabled; // ← chỉ đổi field private
    TelexResetSyllable();
    InvalidateVisual();
    return;
```

Kết quả: gán `keyboard.TelexEnabled = x` từ bên ngoài (code-behind) chỉ đổi

`TelexEnabledProperty` (không ảnh hưởng gì tới UI/hành vi gõ vì UI đọc `_telexEnabled`, không đọc property!) — và bấm phím thật chỉ đổi `_telexEnabled` (không ảnh hưởng `TelexEnabledProperty` nên host app không bao giờ nhận được `PropertyChanged`).

Hai chiều đều "hoạt động nhìn bằng mắt" (vì UI đọc đúng field nội bộ khi user bấm) nên rất dễ lầm tưởng property public đã được wire đúng — chỉ lộ ra khi có code NGOÀI control cần *quan sát* trạng thái (persistence, logging, đồng bộ với view khác).

Giải pháp

Xoá field private trùng lặp, dùng property public (`StyledProperty`) làm NGUỒN DUY NHẤT

ở mọi nơi trong chính control — kể cả logic nội bộ (rendering, xử lý phím):

```
// Xoá: private bool _telexEnabled = false;

// DrawKey: dùng property thay field
bool telexActive = val == "TELEX" && TelexEnabled;
string lbl = val == "TELEX" ? (TelexEnabled ? "VIE" : "ENG") : ...;

// ProcessKey: gọi qua setter public — tự SetValue + TelexResetSyllable + InvalidateVisual
case "TELEX":
    TelexEnabled = !TelexEnabled;
    return;

// default case (đang gõ ký tự):
if (TelexEnabled && val.Length == 1 && char.IsLetter(val[0])) TypeTelex(val);
```

Thêm guard trong setter để tránh set trùng giá trị (`SetValue` vẫn no-op nhưng tránh gọi lại

`TelexResetSyllable()/InvalidateVisual()` không cần thiết khi host code set lại giá trị cũ):

```
set { if (GetValue(TelexEnabledProperty) == value) return; SetValue(...); ... }
```

Áp dụng lại (How to reuse)

- Khi 1 **Control** custom (Avalonia hay bất kỳ framework binding-property nào) có cả `StyledProperty/DependencyProperty` public LÃN 1 field private cho "cùng" khái niệm trạng thái → **kiểm tra xem 2 biến đó có thực sự đồng bộ 2 chiều hay không**. Grep tên field trong toàn bộ file, đối chiếu với tên property — nếu thấy field xuất hiện ở nhánh xử lý input (pointer/key handler) mà property chỉ xuất hiện ở public getter/setter riêng lẻ → nghi ngờ ngay.
- Test bằng mắt (bấm nút, thấy UI đổi đúng) KHÔNG đủ để xác nhận property public đã wire đúng — phải test bằng cách subscribe `PropertyChanged` từ NGOÀI control và log ra, hoặc set breakpoint trong `OnPropertyChanged/setter`, rồi bấm nút thật trên UI để xác nhận nó thực sự fire.
- Bài học rộng hơn: nguyên tắc "1 khái niệm trạng thái = 1 nguồn sự thật" áp dụng ngay cả bên TRONG 1 class — không chỉ giữa các class/layer khác nhau.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Đây là control DÙNG CHUNG (`KztekComponentAvalonia`, qua `ProjectReference`) — sửa ảnh hưởng MỌI project tham chiếu (không chỉ MolexAPP). Đã build lại cả `KztekComponentAvalonia` và project host để xác nhận không có lỗi biên dịch ở phía dùng chung.
- ⚠ Liên quan trực tiếp lesson `[[kzkeyboard-telex-uye-cluster-and-language-label-swapped]]` — lesson đó sửa LABEL hiển thị sai chiều; lesson này là 1 tầng sâu hơn — chính STATE điều khiển label đó tồn tại ở 2 nơi không đồng bộ. Khi sửa 1 trong 2 lesson, nên đọc luôn lesson còn lại.
- ⚠ Trước khi tin "property public đã đúng" chỉ vì build pass + UI test bằng mắt OK — with control có internal rendering riêng (không dùng data-binding chuẩn `{Binding}` mà tự vẽ bằng `Render()`/`DrawingContext`), rủi ro "2 nguồn trạng thái" cao hơn nhiều so với control chuẩn.

Tham chiếu

- File: `KztekComponentAvalonia/Controls/KzKeyboard.cs` (property `TelexEnabled`, field `_telexEnabled` đã xoá, `ProcessKey` case "TELEX", `DrawKey`)
- Project liên quan: MolexAPP (MolexKioskApp) — phát hiện khi thêm tính năng lưu/tự-load ngôn ngữ bàn phím qua `Services/KeyboardLanguageService.cs`; component dùng chung
`E:\KZTEK\Code_Git\5.BaseUI\KztekComponentAvalonia`